

The Sleeping Barber Problem

The Sleeping Barber Problem

- Another classical IPC problem takes place in a barber shop.
- The barber shop has **one barber**, **one barber chair**, and **n chairs for waiting customers**, if any, to sit on.
- If there are **no customers** present, the barber sits down in the **barber chair** and **falls asleep**.
- When a **customer arrives**, he has to **wake up** the sleeping **barber**.
- If additional **customers** arrive while the barber is cutting a customer's hair, they **either sit down** (if there are empty chairs) or **leave the shop** (if all chairs are full).

The Sleeping Barber Problem

- The problem is to program the barber and the customers without getting into race conditions.
- This problem is similar to various queuing situations, such as a multi person helpdesk with a computerized call waiting system for holding a limited number of incoming calls.

Solution

- The solution uses **three semaphores**
 - ***customers***, which counts waiting customers (excluding the customer in the barber chair, who is not waiting),
 - ***barbers***, the number of barbers (0 or 1) who are idle, waiting for customers, and
 - ***mutex***, which is used for mutual exclusion.
- Solution uses a **variable, *waiting***, which also counts the waiting customers.
- The reason for having *waiting* is that there is no way to read the current value of a semaphore.

Barber Process

- When the barber shows up for work in the morning, he executes the procedure *barber*, causing him to *block on the semaphore customers* because it is initially 0.
- The barber then goes to sleep.
- He stays asleep until the first customer shows up.

Barber Process

```
semaphore customers = 0;
semaphore barbers = 0;
semaphore mutex = 1;
int waiting = 0;

void barber(void)
{
    while (TRUE) {
        down(&customers);
        down(&mutex);
        waiting = waiting - 1;
        up(&barbers);
        up(&mutex);
        cut_hair();
    }
}
```

Customer Process

- When a customer arrives, he executes *customer*, starting by acquiring *mutex* to enter a critical region.
- If another customer enters shortly thereafter, the second one will not be able to do anything until the first one has released *mutex*.
- *The customer then checks to see if the number of waiting customers is less than the number of chairs.*
- If not, he releases *mutex* and leaves without a haircut.

Customer Process

- If there is an available chair, the customer increments the integer variable, *waiting*.
- Then he does an *Up on the semaphore customers*, thus waking up the barber.
- ***At this point, the customer and the barber are both awake.***
- When the customer releases *mutex*, the barber grabs it, does some housekeeping, and begins the haircut.
- When the haircut is over, the customer exits the procedure and leaves the shop.

Customer Process

```
void customer(void)
{
    down(&mutex);
    if (waiting < CHAIRS) {
        waiting = waiting + 1;
        up(&customers);
        up(&mutex);
        down(&barbers);
        get_haircut();
    } else {
        up(&mutex);
    }
}
```

Assignment: Dining Philosophers' Problem

```
/* program    diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
    while (true) {
        think();
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal(fork [(i+1) mod 5]);
        signal(fork[i]);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1),
              philosopher (2),    philosopher (3),
              philosopher (4));
}
```

Figure 6.12 A First Solution to the Dining Philosophers Problem

Assignment: Dining Philosophers' Problem

```
/* program    diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int i)
{
    while (true) {
        think();
        wait (room);
        wait (fork[i]);
        wait (fork [(i+1) mod 5]);
        eat();
        signal (fork [(i+1) mod 5]);
        signal (fork[i]);
        signal (room);
    }
}
void main()
{
    parbegin (philosopher (0), philosopher (1),
              philosopher (2),  philosopher (3),
              philosopher (4));
}
```